

PRESENTAZIONE ARCHITETTURA

Gruppo 19 – aLittleByte

Team: Bellet Eleonora, Belluz Diego, Gastaldello Angelica, Kaneda Fernandes Vinicius Yuri, Maule Angela, Oloni Alex, Soligo Leonardo



Obiettivo del progetto

Capitolato C5 /Nexum — proponente Eggon S.r.l.

Il progetto si divide in due ambiti funzionali principali:

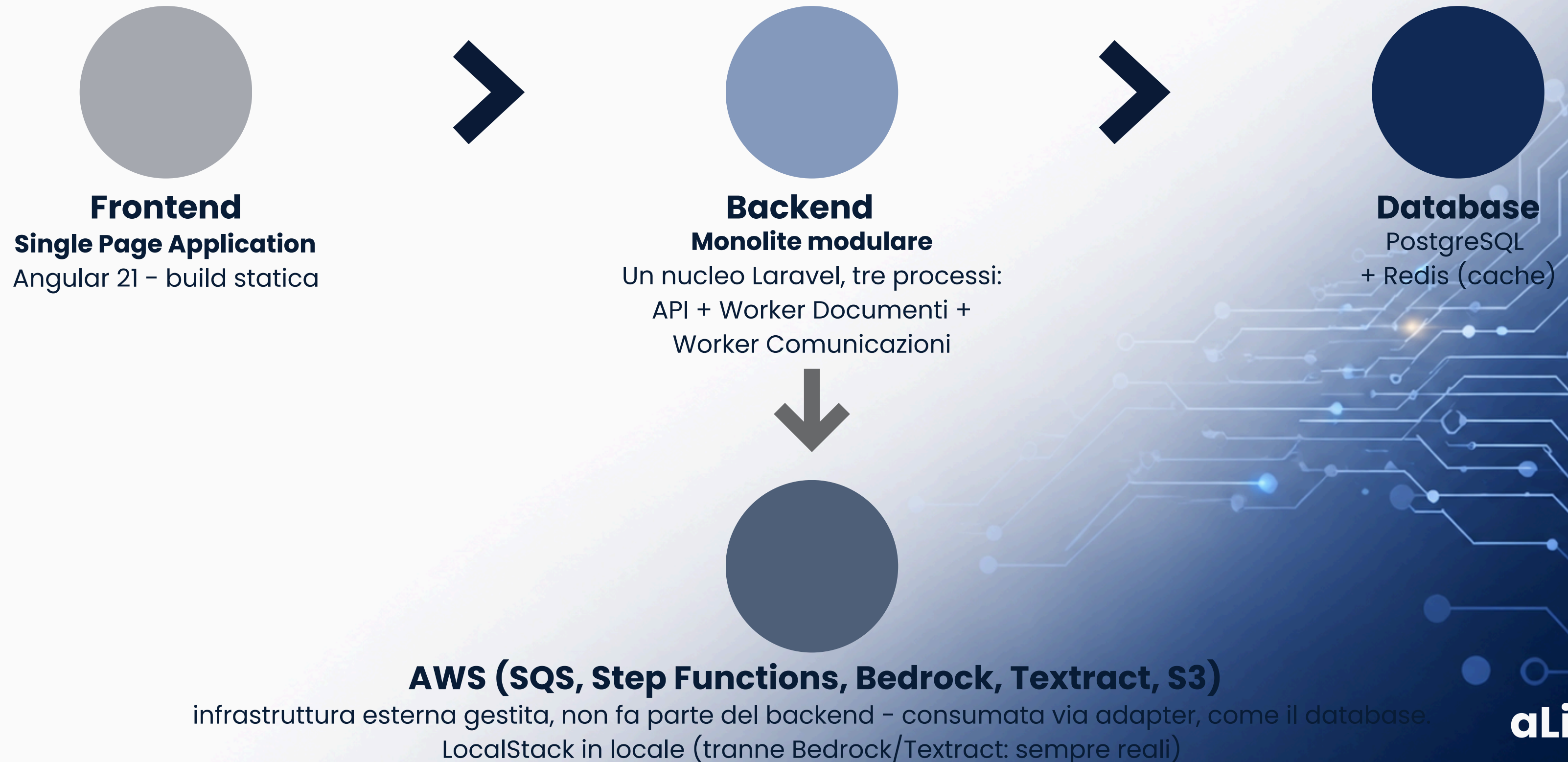
AI Assistant Generativo:

→ redattori aziendali: generazione di comunicazioni interne (titolo, testo, copertina) a partire da un prompt

AI Co-Pilot:

→ consulenti del lavoro: analisi automatica della documentazione HR (split, OCR, riconoscimento destinatario)

Architettura di deployment



Contratto API

La comunicazione tra frontend e backend avviene tramite **API REST**, definite in un contratto OpenAPI condiviso.

GET	/api/v1/documents
POST	/api/v1/documents/ocr
GET	/api/v1/documents/{id}/stream
PUT	/api/v1/documents/{id}/extracted-data
POST	/api/v1/documents/{id}/review
DELETE	/api/v1/documents/{id}

Route del modulo Documents (AI Co-Pilot)

GET	/api/v1/communications
POST	/api/v1/communications
POST	/api/v1/communications/{id}/regenerate
PUT	/api/v1/communications/{id}
POST	/api/v1/communications/{id}/rating
DELETE	/api/v1/communications/{id}

Route del modulo Communications (AI Assistant)

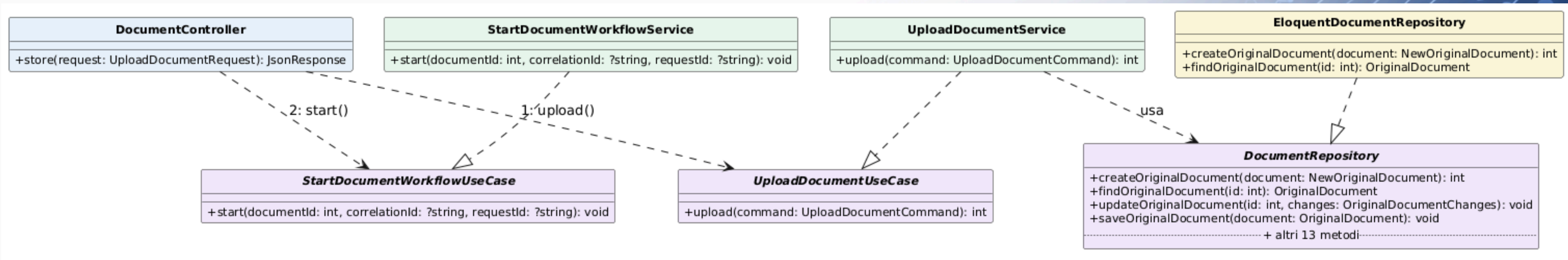
Architettura logica del backend

PROBLEMA:

- La logica di business dipende dai dettagli tecnici: framework, database, SDK AWS.
- Difficile da testare senza avviare l'infrastruttura reale (container, DB, LocalStack).
- Difficile sostituire un componente tecnico senza toccare il dominio.

SOLUZIONE: Architettura Esagonale

- La logica di business è isolata dai dettagli tecnici, non li conosce.
- Comunica con l'esterno solo tramite **Ports** (interfacce) e **Adapters** che le implementano.
- Esempio reale sotto: il flusso di upload, dal Controller fino al repository.



○ Adapter primario — riceve uno stimolo esterno, nessuna logica di business

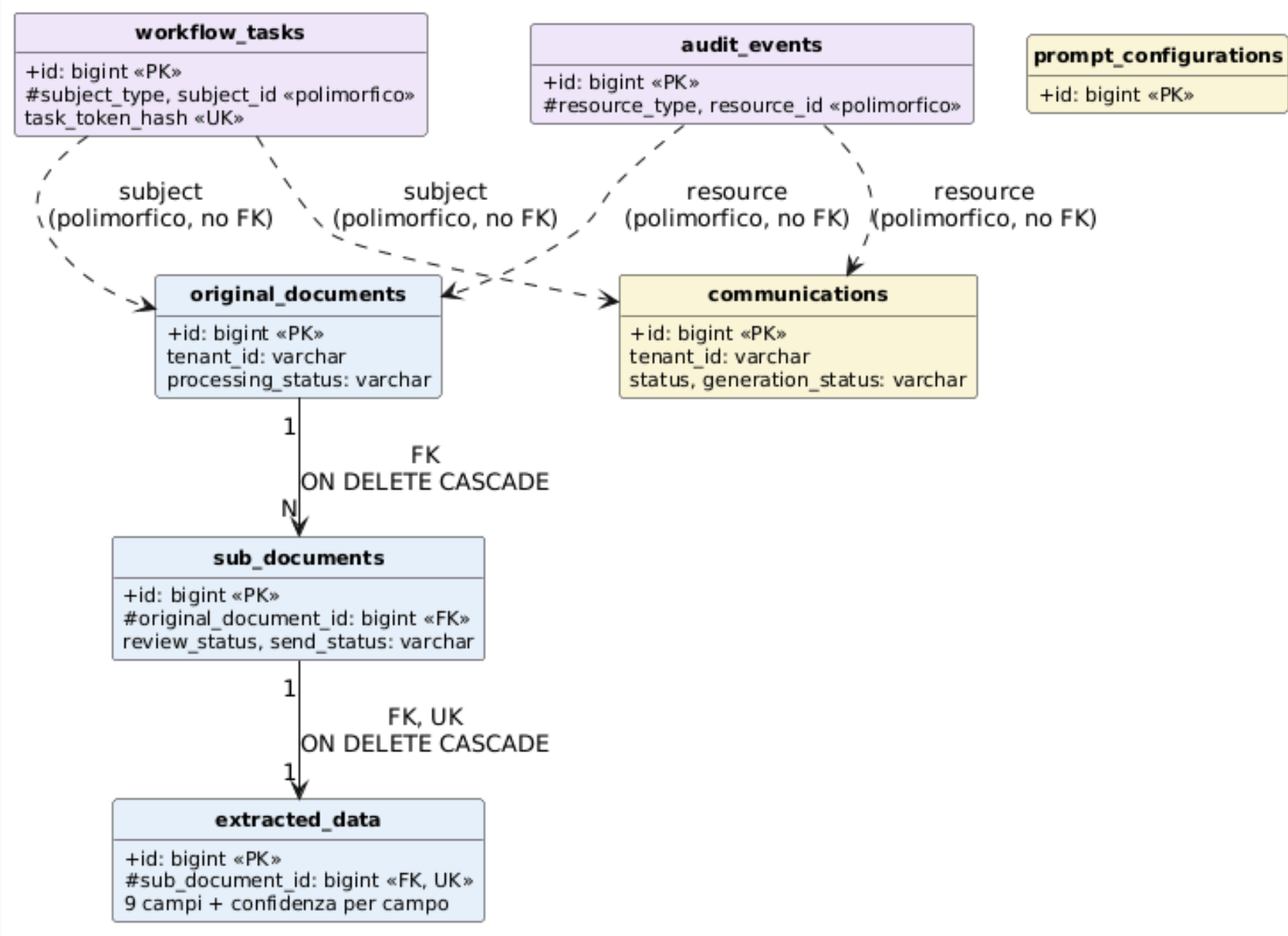
○ Porta — interfaccia, inbound o outbound, definisce una capacità senza implementarla

○ Dominio / caso d'uso — regole di business, istanziabile con new

○ Adapter secondario — implementa la porta verso un sistema esterno

Schema di persistenza dei dati

Il modello è relazionale su PostgreSQL, 7 tabelle. Le tabelle del dominio Documents (*original_documents* → *sub_documents* → *extracted_data*) seguono chiavi esterne classiche con cascade. *workflow_tasks* e *audit_events*, condivise da entrambi i domini, usano invece un riferimento **polimorfico** (coppia tipo/id).



Perché non una foreign key

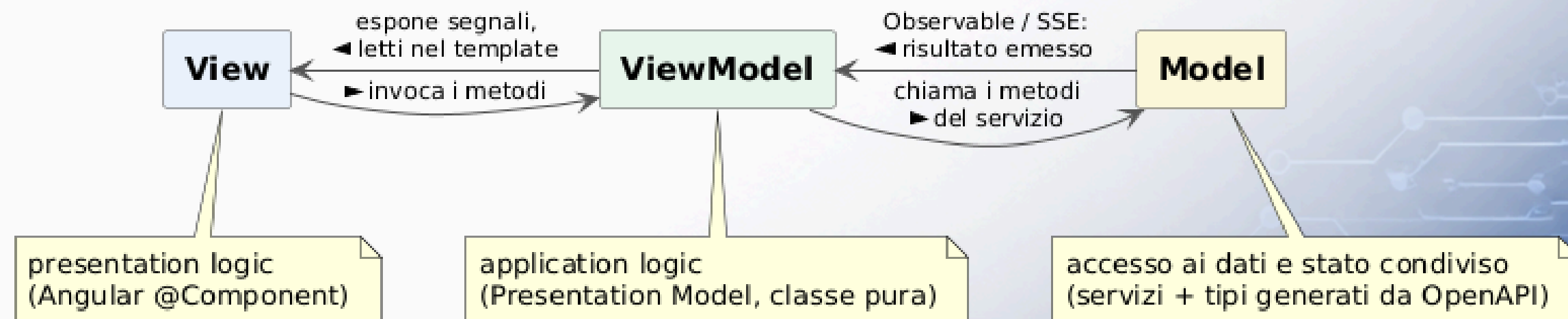
Una tabella condivisa da due domini non può puntare con un unico vincolo a "il documento o la comunicazione, a seconda del caso" — il database non esprime quel tipo di riferimento. Il prezzo di questa scelta: l'integrità di quel riferimento non è garantita dal database, ma verificata a livello applicativo.

Frontend — MVVM

PROBLEMA:

- La logica di presentazione resta legata al componente e quindi al framework che lo costruisce.
- Verificare il comportamento di una pagina richiede di montare la pagina, con il suo ciclo di vita e il DOM.

SOLUZIONE: architettura MVVM, con tre responsabilità separate



View

Il componente e il suo template.
Passiva: riceve dati già derivati e
notifica le azioni, senza deciderle.

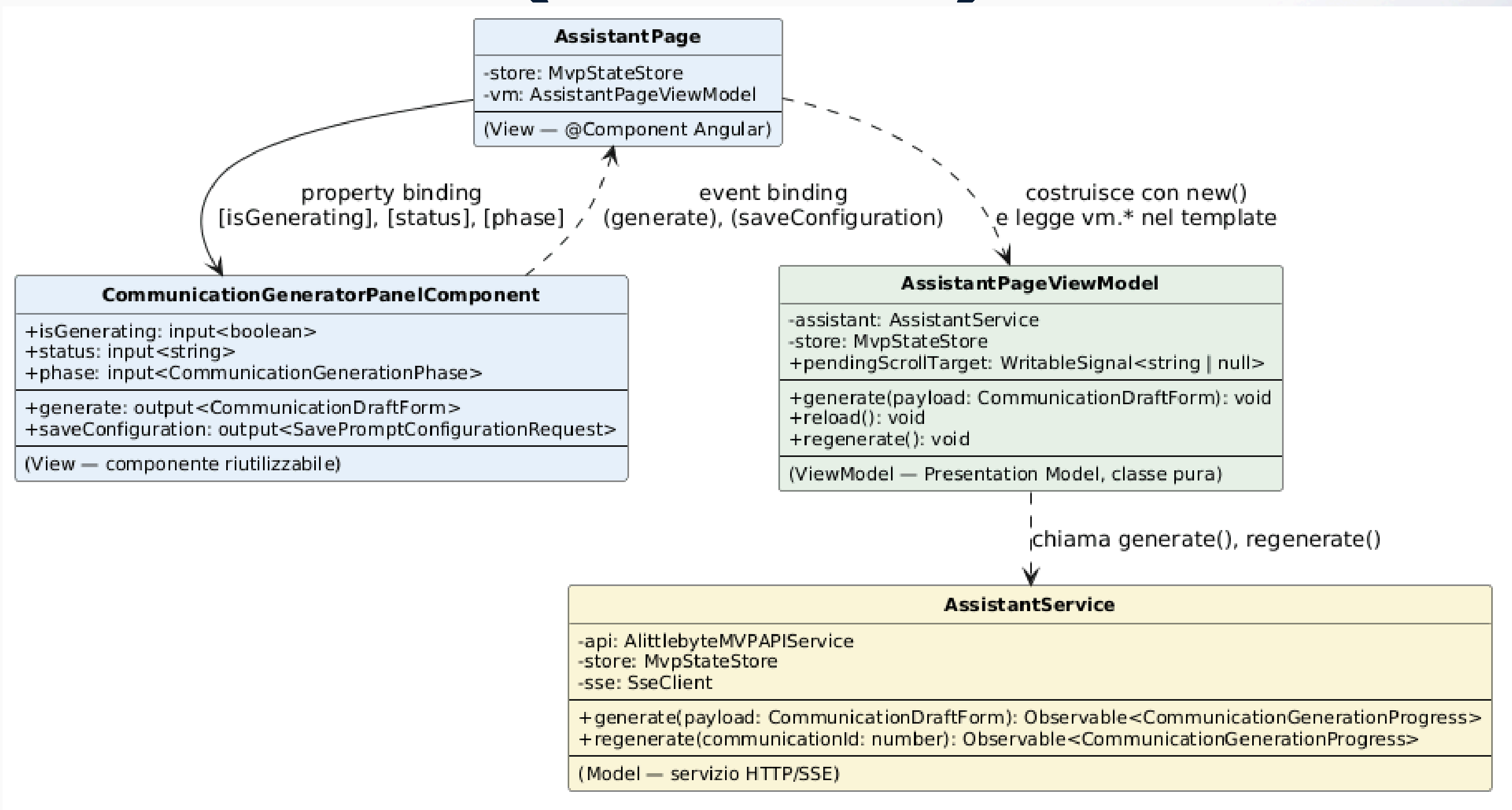
ViewModel

Classe TypeScript senza decoratori
e fuori dall'injector, costruita con
new: non conosce la View.

Model

I servizi che incapsulano la
comunicazione con il backend e
lo stato condiviso: gli unici a
conoscere l'HTTP.

UML Assistant (frontend)



Design Pattern — *Dependency Injection*

PROBLEMA

Un caso d'uso ha bisogno di un repository, di un gateway AI, di uno storage — **porte** astratte (interfacce). Se dovesse costruirsi da solo l'implementazione concreta (*`new EloquentDocumentRepository()`*), diventerebbe legato a quella classe: cambiarla richiederebbe toccare ogni caso d'uso che la usa.

SOLUZIONE

Dependency Injection: le classi non costruiscono le proprie dipendenze, le ricevono già pronte dal costruttore.

Il **Service Container** di Laravel decide da solo cosa passare; un unico punto (*`AppServiceProvider`*) stabilisce quale adapter soddisfa quale porta.

```
// AppServiceProvider.php — porta → adapter
$this->app->singleton(
```

```
    DocumentRepository::class,
```

```
    EloquentDocumentRepository::class
```

```
);
$this->app->singleton(
```

```
    DocumentAiGatewayPort::class,
```

```
    BedrockDocumentAiAdapter::class
```

```
);
// stesso schema per ogni porta secondaria
```

FRONTEND: DOVE LA DI NON SI USA

Angular funziona allo stesso modo per i componenti: *AssistantPage* chiede *AssistantService* con *inject()*, e Angular lo procura dal proprio container.

Ma il **ViewModel** è un'eccezione voluta: costruito a mano con *`new AssistantPageViewModel(...)`*, non dal container.

Perché: un *ViewModel* risolto da Angular richiederebbe *TestBed* per ogni test; costruito con *new* basta passargli dipendenze finte in un test qualsiasi.

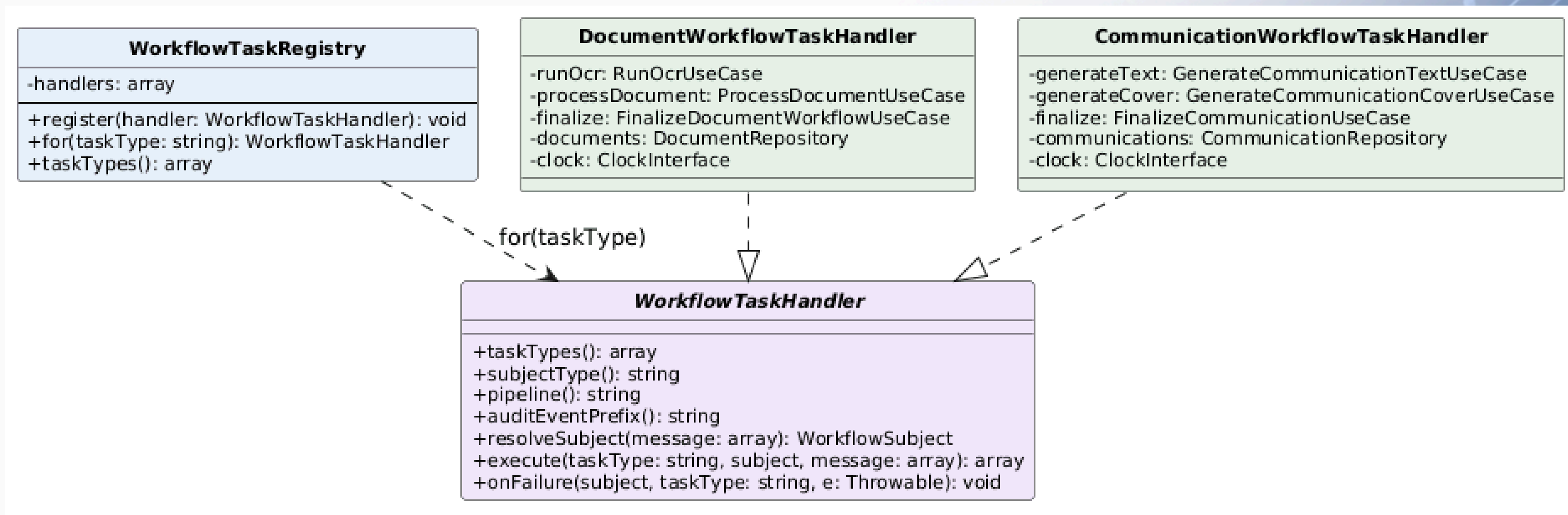
Design Pattern — *Strategy*

PROBLEMA

WorkflowTaskRunner riceve un task da SQS/Step Functions e deve eseguire codice diverso a seconda del **dominio** (Documents o Communications) — restando aperto ad aggiungere un terzo dominio in futuro senza toccare l'orchestrazione esistente (Open/Closed Principle).

SOLUZIONE

Ogni handler concreto implementa lo stesso metodo pubblico *execute()*, chiamato in modo **polimorfico** dopo che *WorkflowTaskRegistry::for(taskType)* lo seleziona a runtime: disaccoppiamento ed estensibilità, nessun *if/switch* sul dominio.



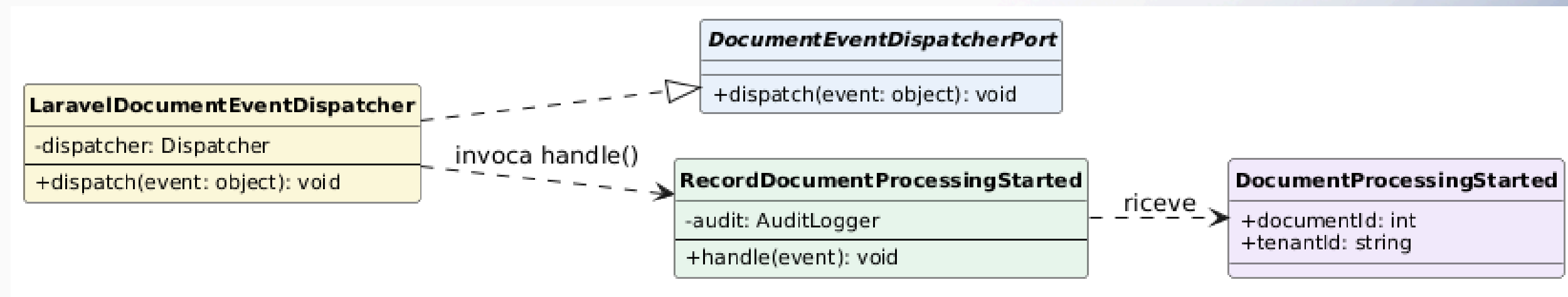
Design Pattern — Observer

PROBLEMA

Un caso d'uso pubblica un evento di dominio (upload, revisione, generazione...). Se dovesse anche scrivere l'audit trail, ogni nuova reazione richiederebbe modificare ogni caso d'uso che genera quell'evento.

SOLUZIONE

Il caso d'uso dispatcha solo l'evento sulla propria porta; un **Listener** separato reagisce in modo indipendente, senza che il chiamante lo conosca.



Perché non tutti gli eventi hanno un listener: nella maggior parte dei casi serve, perché scrive qualcosa che il caso d'uso non scrive da sé (l'audit). Ma per il completamento della pipeline (DocumentWorkflowCompleted) il caso d'uso salva già lo stato finale nel database prima di dispatchare l'evento: non resta nulla da far scrivere a un listener, e infatti non ne è stato registrato uno.

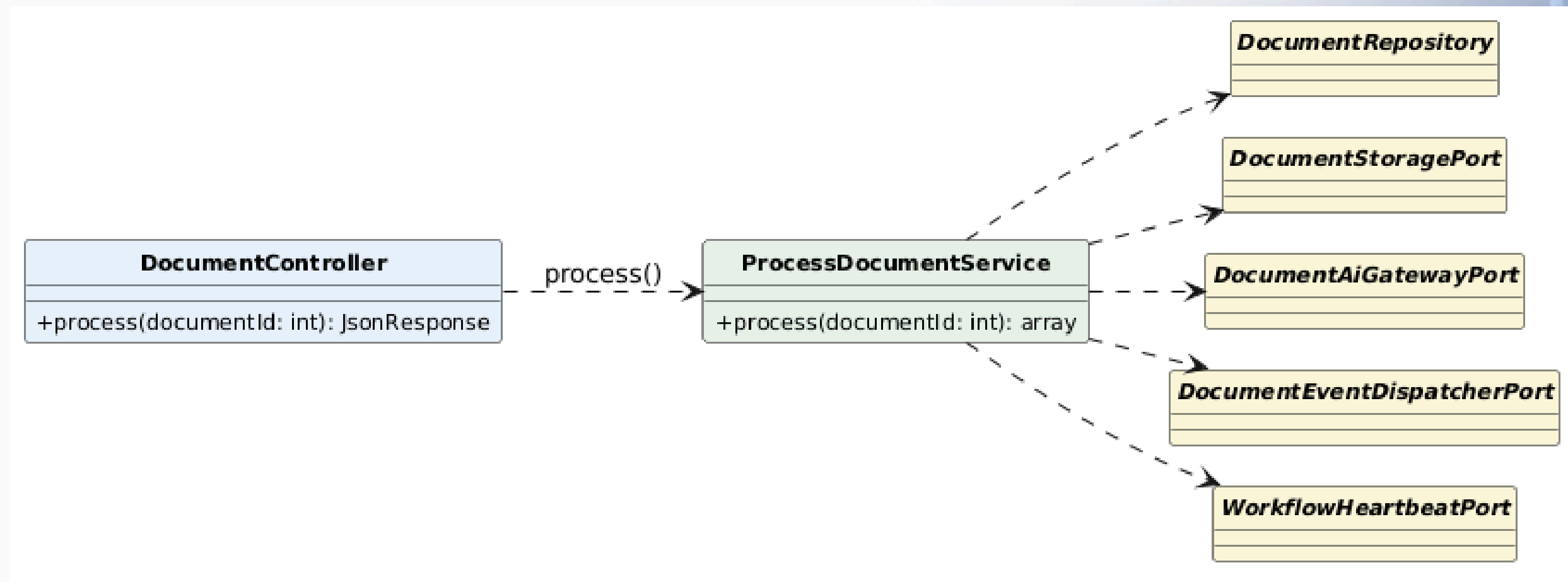
Design Pattern — *Facade*

PROBLEMA

Un caso d'uso come *ProcessDocumentService* deve coordinare 5 collaboratori — repository, storage, gateway AI, dispatcher di eventi, heartbeat — per completare un'unica operazione. Se l'adapter primario che lo invoca dovesse orchestrarli lui stesso, dipenderebbe da dettagli che non gli competono.

SOLUZIONE

Facade: il caso d'uso espone un solo metodo pubblico (*process()*) che nasconde l'orchestrazione interna. Chi lo chiama non sa — né deve sapere — quante porte servono per soddisfarlo.



Design Pattern — *Singleton*

PROBLEMA

Il client Bedrock è costoso da costruire (connessione, credenziali) ed è usato sia da Documents (split/estrazione) che da Communications (generazione testo/immagine): costruirne uno per ogni adapter sprecherebbe connessioni.

SOLUZIONE

Singleton: il Service Container garantisce un'unica istanza di BedrockService per request, condivisa da entrambi i domini — bindata all'interfaccia, non alla classe concreta.

